LEMBAR SOAL PRAKTIKUM
MODUL 07: AJAX DAN KONSUMSI API

Nama: Hafizh Arqamilandri Wakhyudi NIM: 103122400044

Kelas: SE-08-02 Tanggal: 05, November 2026

Kerjakan dua soal proyek berikut. Kedua program secara bersama-sama mencakup seluruh materi Modul 07. Gunakan data contoh dan jalankan melalui browser.

Cakupan dan bobot

Proyek	Materi yang diterapkan	Bobot
Soal 1	Konsep dan alur AJAX; event handling; XMLHttpRequest; Fetch; API lokal/eksternal; loading dan error.	50 poin
Soal 2	Axios GET dan POST; loading/error; useState dan useEffect pada React.	50 poin

Persiapan dan cara menjalankan

1. Gunakan Paket_Praktikum_AJAX.zip dari modul sebelumnya. Ekstrak seluruh paket dan pastikan Node.js 18 atau lebih baru tersedia.
2. Di dalam paket, buat latihan-praktikan/soal-01 dan latihan-praktikan/soal-02. Simpan kode masing-masing proyek pada folder tersebut.
3. Jalankan JALANKAN.bat atau node server.cjs dari folder paket. Biarkan terminal berjalan.
4. Buka http://127.0.0.1:3007/latihan-praktikan/soal-01/ atau alamat soal-02/. Sesuaikan port jika server memakai port lain.

API dan data yang digunakan

Metode dan URL	Keterangan
GET /api/users	Array tiga pengguna: id, name, email.
GET /api/users?mode=empty	HTTP 200 dengan array kosong.
GET /api/users?mode=error	HTTP 503 untuk simulasi gagal.
POST /api/users	Kirim JSON name dan email; HTTP 201, id 101, simulated true. Data tidak disimpan.
GET https://jsonplaceholder.typicode.com/users	API eksternal; membutuhkan internet.

Tambahkan parameter delay dalam milidetik, maksimal 5000. Contoh: /api/users?delay=1500 atau /api/users?mode=error&delay=1000. Mock lokal dapat dipakai tanpa internet.

Ketentuan umum

Simpan data yang ditampilkan berdasarkan response. Tampilkan pesan yang mudah dipahami, gunakan label input, dan tangani kegagalan. Referensi kode boleh memakai contoh pada modul. Skenario uji dan ketentuan pengumpulan tersedia pada halaman 4.

Soal 01. Portal informasi praktikum

Materi: AJAX, event, XMLHttpRequest, Fetch, API lokal/eksternal | Bobot: 50 poin | Folder: soal-01

Laboratorium membutuhkan portal untuk membaca pengumuman dan daftar peserta tanpa memuat ulang seluruh halaman. Buat satu halaman HTML dan JavaScript dengan dua bagian yang saling melengkapi: pengumuman dan direktori peserta.

Berkas yang dibuat

Buat index.html, app.js, dan pengumuman.txt di folder soal-01. Isi pengumuman.txt sebagai berikut:

```
PENGUMUMAN PRAKTIKUM
Praktikum dimulai pukul 08.00.
Bawa kartu praktikum dan rapikan komputer setelah selesai.
```

Kebutuhan program

1. Sediakan kolom Catatan praktikan, tombol Tambah hitungan, angka awal 0, dan waktu halaman pertama dibuka. Catatan, hitungan, dan waktu awal harus tetap ada setelah request selesai.
2. Pada bagian pengumuman, sediakan tombol Muat pengumuman. Event klik harus meminta pengumuman.txt menggunakan XMLHttpRequest secara asynchronous. Periksa readyState dan status HTTP, lalu tampilkan responseText pada elemen pre.
3. Pada bagian direktori, sediakan pilihan sumber Mock lokal atau JSONPlaceholder dan tombol Ambil peserta. Gunakan Fetch untuk mengambil data dari URL sesuai pilihan.
4. Untuk mock, gunakan /api/users?delay=1500. Untuk sumber eksternal, gunakan URL JSONPlaceholder pada halaman 1. Periksa response.ok sebelum membaca JSON.
5. Tampilkan maksimal tiga pengguna pertama beserta emailnya, nama sumber, dan jumlah data yang ditampilkan. Gunakan data dari response; jangan mengisi nama peserta secara manual.
6. Saat menunggu, tampilkan pesan loading dan nonaktifkan tombol request yang sedang berjalan. Tombol Tambah hitungan tetap harus dapat digunakan. Aktifkan kembali tombol request setelah proses selesai.
7. Tampilkan pesan gagal untuk file tidak ditemukan, HTTP gagal, atau gangguan jaringan. Jika API eksternal gagal, arahkan pengguna untuk memeriksa internet atau memilih mock secara manual.

Hasil akhir yang diharapkan

Pengumuman dan direktori dapat diperbarui tanpa reload. Praktikan masih dapat mengetik catatan serta menambah hitungan ketika data sedang diminta. Hasil pengumuman berasal dari XHR, sedangkan daftar peserta berasal dari Fetch.

Penjelasan singkat dalam laporan

Jelaskan event pemicu, request yang dikirim, response yang diterima, dan elemen DOM yang berubah. Tunjukkan bukti bahwa interaksi hitungan tetap berjalan selama menunggu data.

Soal 02. Aplikasi registrasi peserta React

Materi: Axios GET/POST, loading/error, useState, useEffect | Bobot: 50 poin | Folder: soal-02

Buat satu aplikasi React untuk menampilkan daftar peserta dan mengirim formulir pendaftaran. Daftar dimuat otomatis, sedangkan data pendaftaran dikirim setelah pengguna menekan tombol.

Berkas dan library

Buat index.html dan app.jsx di folder soal-02. Siapkan elemen root pada body. Muat React, ReactDOM, Axios, dan Babel berurutan dari ../../vendor/, memakai nama file yang sama dengan contoh React pada modul. Muat app.jsx dengan tag berikut; pemasangan ReactDOM.createRoot() dilakukan di app.jsx.

```
<script type="text/babel" data-presets="react" src="app.jsx"></script>
```

Kebutuhan program

1. Gunakan useState untuk data peserta, loading, error, input formulir, dan status pengiriman. Gunakan useEffect untuk GET awal dan saat pilihan skenario berubah.
2. Ambil daftar memakai axios.get() dan baca response.data. Tampilkan nama serta email menggunakan key dari user.id. Sediakan tombol Muat ulang agar request dapat dijalankan lagi.
3. Sediakan pilihan Berhasil, Data kosong, Error HTTP, dan Timeout. Gunakan URL serta batas tunggu pada tabel di bawah. Tampilkan loading, hasil, kosong, atau pesan gagal sesuai kondisi sebenarnya.
4. Rapikan effect saat pilihan berubah atau komponen dilepas, misalnya dengan AbortController dan penanda effect aktif. Response lama tidak boleh menimpa hasil pilihan terbaru.
5. Sediakan form Nama, Email, dan tombol Kirim pendaftaran. Cegah reload dengan preventDefault(), tolak nama kosong atau hanya spasi, dan validasi format email.
6. Kirim objek name dan email memakai axios.post("/api/users", objekData). Nonaktifkan tombol saat mengirim. Tampilkan status HTTP, id, nama, email, simulated, serta pesan Simulasi berhasil, data tidak disimpan.
7. Tangani kegagalan GET dan POST, lalu pulihkan kontrol setelah selesai. Pendaftaran tidak menambah data ke server; tampilkan response POST terpisah dari daftar GET.

Skenario GET	URL	Timeout Axios
Berhasil	/api/users?delay=1000	5000 ms
Data kosong	/api/users?mode=empty&delay=1000	5000 ms
Error HTTP	/api/users?mode=error&delay=1000	5000 ms
Timeout	/api/users?delay=2500	700 ms

Gunakan try, catch, dan finally untuk mengelola kondisi proses. Batas waktu 700 ms harus menghasilkan kegagalan timeout sebelum response yang dijeda 2500 ms tiba.

Pengujian, pengumpulan, dan penilaian

Jalankan skenario berikut

Soal	Pengujian	Hasil yang harus terlihat
1	Muat pengumuman, lalu ubah isi file dan muat lagi.	Isi baru tampil melalui XHR.
1	Ubah sementara URL pengumuman menjadi hilang.txt.	Pesan gagal dengan status 404. Kembalikan URL setelah uji.
1	Ambil mock sambil mengetik dan klik hitungan tiga kali.	Tiga peserta tampil; catatan tetap ada, hitungan bertambah, waktu awal tetap sama.
1	Pilih API eksternal dan kirim request.	Maksimal tiga pengguna tampil atau pesan gagal yang jelas. Sumber tidak berganti otomatis.
2	Buka halaman, lalu klik Muat ulang.	GET berjalan otomatis saat awal; klik memicu request baru.
2	Coba Data kosong dan Error HTTP.	Pesan kosong atau HTTP 503; daftar lama dibersihkan.
2	Coba Timeout, lalu kembali ke Berhasil.	Request timeout pada batas 700 ms; aplikasi dapat digunakan lagi.
2	Ganti pilihan ketika request masih berjalan.	Hasil akhir mengikuti pilihan terbaru.
2	Kirim Rina Praktikan, rina@example.com.	HTTP 201; id 101; data sesuai input; simulated true. GET tetap tiga peserta.
2	Nama hanya spasi atau format email tidak valid.	Form menolak pengiriman dan memberi petunjuk perbaikan.

Jika internet atau layanan eksternal tidak tersedia, catat kondisi tersebut. Penanganan kegagalan menjadi bukti uji jalur eksternal. API lokal dan POST di atas tetap dapat diuji tanpa internet.

Berkas yang dikumpulkan

Kumpulkan NIM_Nama_Modul07.zip berisi folder soal-01, soal-02, README cara menjalankan pada paket awal, dan laporan hasil uji. Sertakan tangkapan layar kedua program, bukti POST, serta satu kondisi gagal. Catat hasil aktual setiap skenario dan jelaskan alur XHR, Fetch, Axios, serta peran state/effect React secara singkat.

Rubrik penilaian (nilai akhir maksimal 100 poin)

Aspek	Soal 1	Soal 2
Fungsi utama sesuai kebutuhan	30	30
Loading, validasi, dan penanganan kondisi	10	10
Keterbacaan kode dan bukti pengujian	10	10
Jumlah	50	50

JAWABAN

Soal 1

index.html

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>Soal 01 - Portal Informasi Praktikum | Praktikum AJAX</title>
  <link rel="icon" href="data:,">
  <link rel="stylesheet" href="../../assets/style.css">
</head>
<body>
  <a class="skip" href="#utama">Lewati navigasi</a>

  <aside class="sidebar">
    <a class="brand" href="../../index.html"><span class="brand-mark"
aria-hidden="true">{ }</span> Praktikum AJAX</a>
    <div class="scope">Modul 07 / Pemrograman Web</div>
    <nav aria-label="Daftar materi">
      <a href="../../index.html">Peta materi</a>
      <a href="../../01-apa-itu-ajax/index.html">01 Apa itu AJAX?</a>
      <a href="../../02-cara-kerja-ajax/index.html">02 Cara kerja
AJAX</a>
      <a href="../../03-event-xhr/index.html">03 Event dan
XMLHttpRequest</a>
      <a href="../../04-fetch-get/index.html">04 GET dengan Fetch</a>
```

```
      <a href="../../05-axios-get/index.html">05 GET dengan Axios</a>
      <a href="../../06-axios-post/index.html">06 POST dengan Axios</a>
      <a href="../../07-loading-error/index.html">07 Loading dan error
handling</a>
      <a href="../../08-react-state/index.html">08 Loading dan error di
React</a>
      <a href="../../09-api-eksternal/index.html">09 API lokal dan API
eksternal</a>
      <a href="index.html" aria-current="page">Soal 01 - Portal
Praktikum</a>
      <a href="../soal-02/index.html">Soal 02 - Registrasi React</a>
    </nav>
    <p class="sidebar-footer">Belajar dari request pertama sampai state
di React.<br>Data latihan: pengguna fiktif.</p>
  </aside>

  <header class="topbar">
    <span>Modul 07 <span class="context">/ Soal 01: Portal Informasi
Praktikum</span></span>
    <a href="../../Modul_07_AJAX_Sederhana.pdf">Baca modul PDF</a>
  </header>

  <div class="mobile-nav">
    <label class="skip" for="pindah-materi">Pilih materi</label>
    <select id="pindah-materi" aria-label="Pindah materi">
      <option value="../../index.html">Peta materi</option>
      <option value="../../01-apa-itu-ajax/index.html">01 Apa itu
AJAX?</option>
      <option value="../../02-cara-kerja-ajax/index.html">02 Cara kerja
AJAX</option>
      <option value="../../03-event-xhr/index.html">03 Event dan
XMLHttpRequest</option>
      <option value="../../04-fetch-get/index.html">04 GET dengan
Fetch</option>
      <option value="../../05-axios-get/index.html">05 GET dengan
Axios</option>
      <option value="../../06-axios-post/index.html">06 POST dengan
Axios</option>
      <option value="../../07-loading-error/index.html">07 Loading dan
error handling</option>
      <option value="../../08-react-state/index.html">08 Loading dan
error di React</option>
      <option value="../../09-api-eksternal/index.html">09 API lokal dan
API eksternal</option>
      <option value="index.html" selected>Soal 01 - Portal
```

```
Praktikum</option>
      <option value="../soal-02/index.html">Soal 02 - Registrasi
React</option>
    </select>
  </div>

  <main id="utama" class="sheet">
    <div class="file-warning" data-file-warning hidden>
      <strong>Jalankan server terlebih dahulu.</strong><br>
      Ekstrak seluruh ZIP, lalu klik dua kali <code>JALANKAN.bat</code>.
Buka <a href="http://127.0.0.1:3007">http://127.0.0.1:3007</a>.
      Contoh AJAX perlu alamat HTTP, bukan file://. Node.js 18+
diperlukan; Node.js LTS disarankan.
    </div>

    <div class="page-meta">Tugas Praktikum Modul 07 / Soal 01 (50
Poin)</div>
    <h1>Portal Informasi Praktikum</h1>
    <p class="intro">Membaca pengumuman via XMLHttpRequest dan direktori
peserta via Fetch API tanpa memuat ulang halaman. Interaksi catatan dan
hitungan tetap berjalan saat request asynchronous berlangsung.</p>

    <div class="workspace">
      <section class="practice" aria-label="Program portal praktikum">
        <h2>Panel Interaksi dan Pengujian Non-Reload</h2>
        <p class="meta">Waktu halaman pertama dibuka: <strong
id="waktu-buka">-</strong></p>

        <label for="catatan">Catatan praktikan</label>
        <input type="text" id="catatan" placeholder="Ketik catatan di
sini untuk membuktikan halaman tidak reload...">

        <div class="actions">
          <button type="button" class="secondary" id="tombol-hitung">
            Tambah hitungan: <span id="angka-hitungan">0</span>
          </button>
        </div>

        <hr style="border:0;border-top:1px solid
var(--border);margin:24px 0;">

        <h2>1. Bagian Pengumuman (XMLHttpRequest)</h2>
        <p class="meta">Meminta berkas teks <code>pengumuman.txt</code>
secara asynchronous via XHR.</p>
        <button type="button" id="muat-pengumuman">Muat
```

```
pengumuman</button>
        <p id="status-pengumuman" class="result" role="status">Pengumuman
belum dimuat.</p>
        <pre id="isi-pengumuman" class="result"
style="margin-top:10px;">Belum ada pengumuman.</pre>

        <hr style="border:0;border-top:1px solid
var(--border);margin:24px 0;">

        <h2>2. Bagian Direktori Peserta (Fetch API)</h2>
        <label for="pilihan-sumber">Pilihan sumber direktori</label>
        <select id="pilihan-sumber">
          <option value="lokal">Mock lokal
(/api/users?delay=1500)</option>
          <option value="eksternal">JSONPlaceholder
(https://jsonplaceholder.typicode.com/users)</option>
        </select>

        <button type="button" id="ambil-peserta">Ambil peserta</button>
        <p id="status-peserta" class="result" role="status">Daftar
peserta belum diminta.</p>
        <ul id="daftar-peserta" class="users"></ul>
      </section>

      <section class="explanation">
        <h2>Inti Materi dan Petunjuk Pengujian</h2>
        <p>Halaman ini menerapkan dua teknik AJAX sekaligus untuk
membuktikan sifat asynchronous dalam aplikasi web:</p>
        <ol>
          <li><strong>XMLHttpRequest (XHR):</strong> Mengambil berkas
statis <code>pengumuman.txt</code> secara asinkron dengan memeriksa
<code>readyState === 4</code> dan status HTTP 200–299.</li>
          <li><strong>Fetch API:</strong> Mengambil data JSON pengguna
dengan memeriksa <code>response.ok</code>, memotong maksimal 3 pengguna
pertama, dan menampilkan nama sumber beserta jumlah data.</li>
          <li><strong>Non-blocking UI:</strong> Saat request pengumuman
atau peserta sedang berjalan (status loading), praktikan dapat terus
mengetik catatan atau mengklik tombol <em>Tambah hitungan</em>. Nilai
catatan, angka hitungan, dan waktu awal tidak pernah hilang karena
seluruh proses berjalan tanpa memuat ulang (reload) halaman.</li>
        </ol>

        <h3>Skenario Pengujian Soal 1</h3>
        <ul>
          <li><strong>Uji Pengumuman:</strong> Klik tombol <em>Muat
```

```
pengumuman</em>. Ubah isi <code>pengumuman.txt</code> lalu klik lagi; isi baru langsung tampil.</li>
          <li><strong>Uji Error 404:</strong> Ubah sementara target request di <code>app.js</code> menjadi <code>hilang.txt</code>; status HTTP 404 akan ditampilkan secara informatif.</li>
          <li><strong>Uji Asynchronous:</strong> Klik <em>Ambil peserta</em> (Mock lokal dijeda 1,5 detik). Selama loading, ketik di kolom catatan dan klik <em>Tambah hitungan</em> 3 kali. Catatan, angka 3, dan waktu awal tetap utuh saat 3 pengguna muncul.</li>
          <li><strong>Uji API Eksternal:</strong> Pilih <em>JSONPlaceholder</em> lalu klik <em>Ambil peserta</em>. Maksimal 3 pengguna eksternal akan tampil atau muncul panduan jika jaringan terputus. Pilihan sumber tidak berganti otomatis.</li>
        </ul>
      </section>
    </div>

    <details class="source" data-source="app.js">
      <summary>Lihat kode JavaScript: app.js</summary>
      <p class="meta">File: latihan-praktikan/soal-01/app.js. Menangani XHR, Fetch, dan event handling.</p>
      <pre class="source-code"><code>Memuat kode...</code></pre>
    </details>

    <details class="source" data-source="index.html">
      <summary>Lihat kode HTML: index.html</summary>
      <pre class="source-code"><code>Memuat kode...</code></pre>
    </details>

    <details class="source" data-source="pengumuman.txt">
      <summary>Lihat isi berkas: pengumuman.txt</summary>
      <pre class="source-code"><code>Memuat kode...</code></pre>
    </details>

    <div class="pager">
      <a href="../../09-api-eksternal/">&larr; Kembali ke Materi 09</a>
      <a href="../soal-02/">Lanjut ke Soal 02 &rarr;</a>
    </div>
  </main>

  <script src="../../assets/ui.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

app.js

```
// Soal 01: Portal Informasi Praktikum
// Menggabungkan XHR, Fetch, event handling, serta pembuktian sifat
asynchronous tanpa reload.

const inputCatatan = document.querySelector("#catatan");
const tombolHitung = document.querySelector("#tombol-hitung");
const teksAngka = document.querySelector("#angka-hitungan");
const teksWaktuBuka = document.querySelector("#waktu-buka");

let hitungan = 0;

const waktuSekarang = new Date().toLocaleTimeString("id-ID", {
  hour: "2-digit",
  minute: "2-digit",
  second: "2-digit"
});
teksWaktuBuka.textContent = waktuSekarang;

tombolHitung.addEventListener("click", () => {
  hitungan += 1;
  teksAngka.textContent = hitungan;
});

const tombolPengumuman = document.querySelector("#muat-pengumuman");
const statusPengumuman = document.querySelector("#status-pengumuman");
const isiPengumuman = document.querySelector("#isi-pengumuman");

tombolPengumuman.addEventListener("click", () => {

  tombolPengumuman.disabled = true;
  statusPengumuman.dataset.state = "loading";
  statusPengumuman.textContent = "Memuat pengumuman via
XMLHttpRequest...";

  const xhr = new XMLHttpRequest();

  xhr.onreadystatechange = () => {
    if (xhr.readyState !== 4) return;
    if (xhr.status >= 200 && xhr.status < 300) {
      statusPengumuman.dataset.state = "success";
      statusPengumuman.textContent = `Pengumuman berhasil dimuat (HTTP
```

```
${xhr.status}).`;
      isiPengumuman.textContent = xhr.responseText;
    } else if (xhr.status === 404) {
      statusPengumuman.dataset.state = "error";
      statusPengumuman.textContent = "Gagal memuat pengumuman: Berkas
tidak ditemukan (HTTP 404).";
      isiPengumuman.textContent = "Berkas pengumuman tidak ditemukan.";
    } else if (xhr.status !== 0) {
      statusPengumuman.dataset.state = "error";
      statusPengumuman.textContent = `Gagal memuat pengumuman: HTTP
${xhr.status}`;
      isiPengumuman.textContent = `Terjadi kesalahan respons server (HTTP
${xhr.status}).`;
    }
  };

  xhr.onerror = () => {
    statusPengumuman.dataset.state = "error";
    statusPengumuman.textContent = "Gagal memuat pengumuman: Terjadi
gangguan jaringan atau koneksi terputus.";
    isiPengumuman.textContent = "Tidak dapat terhubung ke server.";
  };

  xhr.ontimeout = () => {
    statusPengumuman.dataset.state = "error";
    statusPengumuman.textContent = "Gagal memuat pengumuman: Batas waktu
tunggu habis (timeout).";
    isiPengumuman.textContent = "Waktu tunggu habis.";
  };

  xhr.onloadend = () => {
    tombolPengumuman.disabled = false;
  };

  xhr.open("GET", "pengumuman.txt", true);
  xhr.timeout = 5000;
  xhr.send();
});

// 3. Bagian Direktori Peserta (Fetch API)
const tombolPeserta = document.querySelector("#ambil-peserta");
const pilihanSumber = document.querySelector("#pilihan-sumber");
const statusPeserta = document.querySelector("#status-peserta");
const daftarPeserta = document.querySelector("#daftar-peserta");
```

```
tombolPeserta.addEventListener("click", async () => {
  const sumber = pilihanSumber.value;
  const isLokal = sumber === "lokal";
  const namaSumber = isLokal ? "Mock lokal" : "JSONPlaceholder";
  const url = isLokal ? "/api/users?delay=1500" :
"https://jsonplaceholder.typicode.com/users";

  tombolPeserta.disabled = true;
  statusPeserta.dataset.state = "loading";
  statusPeserta.textContent = `Memuat data peserta dari
${namaSumber}...`;
  daftarPeserta.replaceChildren();

  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), 8000);

  try {
    const response = await fetch(url, { signal: controller.signal });

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const users = await response.json();

    const dataDitampilkan = users.slice(0, 3);

    for (const user of dataDitampilkan) {
      const item = document.createElement("li");
      item.textContent = `${user.name} - ${user.email}`;
      daftarPeserta.append(item);
    }

    statusPeserta.dataset.state = "success";
    statusPeserta.textContent = `Sumber: ${namaSumber} | Menampilkan
${dataDitampilkan.length} data peserta dari total ${users.length}
pengguna.`;
  } catch (error) {
    statusPeserta.dataset.state = "error";
    if (error.name === "AbortError") {
      statusPeserta.textContent = `Gagal: Waktu tunggu habis saat
menghubungi ${namaSumber}.`;
    } else if (!isLokal) {
      statusPeserta.textContent = `Gagal memuat dari API eksternal
(${error.message}). Periksa koneksi internet Anda atau pilih sumber Mock
```

```
lokal secara manual.`;
    } else {
      statusPeserta.textContent = `Gagal memuat dari mock lokal
(${error.message}). Pastikan server lokal masih berjalan.`;
    }
  } finally {
    clearTimeout(timer);
    tombolPeserta.disabled = false;
  }
});
```

Soal 2
app.jsx

```
// Soal 02: Aplikasi Registrasi Peserta React
// Mengimplementasikan Axios GET & POST, useState, useEffect, cleanup
AbortController, dan validasi form.

const { useState, useEffect } = React;

const SKENARIO = {
  berhasil: {
    label: "Berhasil",
    url: "/api/users?delay=1000",
    timeout: 5000
  },
  empty: {
    label: "Data kosong",
    url: "/api/users?mode=empty&delay=1000",
    timeout: 5000
  },
  error: {
    label: "Error HTTP",
    url: "/api/users?mode=error&delay=1000",
    timeout: 5000
  },
  timeout: {
    label: "Timeout",
    url: "/api/users?delay=2500",
    timeout: 700
  }
};
```

```
function AplikasiRegistrasi() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [scenario, setScenario] = useState("berhasil");
  const [percobaan, setPercobaan] = useState(0);

  const [nama, setNama] = useState("");
  const [email, setEmail] = useState("");
  const [formError, setFormError] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [postResult, setPostResult] = useState(null);
  const [postError, setPostError] = useState("");


  useEffect(() => {
    const controller = new AbortController();
    let aktif = true;

    async function ambilData() {
      setLoading(true);
      setError("");
      setUsers([]);

      const config = SKENARIO[scenario];

      try {
        const response = await axios.get(config.url, {
          timeout: config.timeout,
          signal: controller.signal
        });

        if (aktif) {
          setUsers(response.data);
        }
      } catch (err) {
        if (!aktif) return;

        if (axios.isCancel(err) || err.name === "CanceledError") {
          return;
        }

        if (err.code === "ECONNABORTED" || (err.message &&
err.message.toLowerCase().includes("timeout"))) {
          setError(`Timeout: Batas waktu tunggu (${config.timeout} ms)
```

```
terlampaui sebelum server merespons.`);
        } else if (err.response) {
          setError(`Gagal: Server merespons HTTP ${err.response.status}
(${err.response.data?.message || "Kesalahan server"}).`);
        } else {
          setError(`Gagal memuat: ${err.message}`);
        }
      } finally {
        if (aktif) {
          setLoading(false);
        }
      }
    }

    ambilData();

    return () => {
      aktif = false;
      controller.abort();
    };
  }, [scenario, percobaan]);

  async function handleKirimPendaftaran(event) {
    event.preventDefault();

    setFormError("");
    setPostError("");
    setPostResult(null);

    const namaBersih = nama.trim();
    const emailBersih = email.trim();

    if (!namaBersih) {
      setFormError("Nama peserta wajib diisi dan tidak boleh hanya berupa
spasi.");
      return;
    }

    const polaEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!polaEmail.test(emailBersih)) {
      setFormError("Format email tidak valid. Masukkan format yang benar,
contoh: rina@example.com");
      return;
    }
```

```
    setSubmitting(true);

    try {
      const response = await axios.post("/api/users", {
        name: namaBersih,
        email: emailBersih
      }, {
        timeout: 5000,
        headers: { "Content-Type": "application/json" }
      });

      setPostResult({
        status: response.status,
        id: response.data.id,
        name: response.data.name,
        email: response.data.email,
        simulated: response.data.simulated,
        message: "Simulasi berhasil, data tidak disimpan."
      });
    } catch (err) {
      if (err.response) {
        setPostError(`Gagal mengirim (HTTP ${err.response.status}):
${err.response.data?.message || "Terjadi kesalahan server"}.`);
      } else {
        setPostError(`Gagal mengirim: ${err.message}`);
      }
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <section className="practice" aria-busy={loading || submitting}>
      <h2>1. Daftar Peserta Praktikum (GET Axios)</h2>

      <label htmlFor="pilihan-skenario">Pilih skenario response</label>
      <select
        id="pilihan-skenario"
        value={scenario}
        onChange={e => setScenario(e.target.value)}
      >
        <option value="berhasil">Berhasil (/api/users?delay=1000 -
Timeout 5000ms)</option>
        <option value="empty">Data kosong
(/api/users?mode=empty&delay=1000 - Timeout 5000ms)</option>
```

```
        <option value="error">Error HTTP
(/api/users?mode=error&delay=1000 - Timeout 5000ms)</option>
        <option value="timeout">Timeout (/api/users?delay=2500 - Timeout
700ms)</option>
      </select>

      <div className="actions" style={{ marginBottom: "16px" }}>
        <button
          type="button"
          disabled={loading}
          onClick={() => setPercobaan(n => n + 1)}
        >
          Muat ulang
        </button>
      </div>

      <p
        id="status-get"
        className="result"
        role="status"
        data-state={error ? "error" : (loading ? "loading" : "success")}
      >
        {loading
          ? "Memuat daftar peserta..."
          : error
          ? error
          : users.length
          ? `Berhasil memuat ${users.length} pengguna.`
          : "Data kosong (tidak ada peserta)."}
      </p>

      {!loading && !error && users.length > 0 && (
        <ul className="users">
          {users.map(user => (
            <li key={user.id}>
              <strong>{user.name}</strong> &mdash;
<span>{user.email}</span> (ID: {user.id})
            </li>
          ))}
        </ul>
      )}

      <hr style={{ border: 0, borderTop: "1px solid var(--border)",
margin: "24px 0" }} />
```

```
      <h2>2. Formulir Registrasi Peserta (POST Axios)</h2>
      <form onSubmit={handleKirimPendaftaran} noValidate>
        <label htmlFor="nama">Nama Peserta</label>
        <input
          type="text"
          id="nama"
          value={nama}
          onChange={e => setNama(e.target.value)}
          placeholder="Contoh: Rina Praktikan"
          disabled={submitting}
        />

        <label htmlFor="email">Email Peserta</label>
        <input
          type="email"
          id="email"
          value={email}
          onChange={e => setEmail(e.target.value)}
          placeholder="Contoh: rina@example.com"
          disabled={submitting}
        />

        {formError && (
          <p className="result" data-state="error" role="alert" style={{
marginTop: "0", marginBottom: "16px" }}>
            {formError}
          </p>
        )}

        <button type="submit" disabled={submitting}>
          {submitting ? "Mengirim pendaftaran..." : "Kirim pendaftaran"}
        </button>
      </form>

      {/* Bagian respon POST terpisah dari daftar GET */}
      {postResult && (
        <div className="result" data-state="success" style={{ marginTop:
"20px" }}>
          <strong>Hasil Response POST:</strong>
          <p style={{ margin: "6px 0" }}>Status HTTP:
<strong>{postResult.status} Created</strong></p>
          <p style={{ margin: "6px 0" }}>Pesan Server:
<em>{postResult.message}</em></p>
          <div style={{ marginTop: "10px" }}>
            <div><strong>ID Dihasilkan:</strong> {postResult.id}</div>
```

```
            <div><strong>Nama Terkirim:</strong> {postResult.name}</div>
            <div><strong>Email Terkirim:</strong>
{postResult.email}</div>
            <div><strong>Simulated:</strong> {postResult.simulated ?
"true (Data tidak disimpan ke server)" : "false"}</div>
          </div>
        </div>
      )}

      {postError && (
        <p className="result" data-state="error" role="alert" style={{
marginTop: "20px" }}>
          {postError}
        </p>
      )}
    </section>
  );
}

ReactDOM.createRoot(document.querySelector("#root")).render(<AplikasiRegi
strasi />);
```

index.html

```
<!doctype html>
<html lang="id">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>Soal 02 - Registrasi Peserta React | Praktikum AJAX</title>
  <link rel="icon" href="data:,">
  <link rel="stylesheet" href="../../assets/style.css">
</head>
<body>
  <a class="skip" href="#utama">Lewati navigasi</a>

  <aside class="sidebar">
    <a class="brand" href="../../index.html"><span class="brand-mark"
aria-hidden="true">{ }</span> Praktikum AJAX</a>
    <div class="scope">Modul 07 / Pemrograman Web</div>
    <nav aria-label="Daftar materi">
      <a href="../../index.html">Peta materi</a>
      <a href="../../01-apa-itu-ajax/index.html">01 Apa itu AJAX?</a>
      <a href="../../02-cara-kerja-ajax/index.html">02 Cara kerja
AJAX</a>
      <a href="../../03-event-xhr/index.html">03 Event dan
XMLHttpRequest</a>
      <a href="../../04-fetch-get/index.html">04 GET dengan Fetch</a>
      <a href="../../05-axios-get/index.html">05 GET dengan Axios</a>
      <a href="../../06-axios-post/index.html">06 POST dengan Axios</a>
      <a href="../../07-loading-error/index.html">07 Loading dan error
handling</a>
      <a href="../../08-react-state/index.html">08 Loading dan error di
React</a>
      <a href="../../09-api-eksternal/index.html">09 API lokal dan API
eksternal</a>
      <a href="../soal-01/index.html">Soal 01 - Portal Praktikum</a>
      <a href="index.html" aria-current="page">Soal 02 - Registrasi
React</a>
    </nav>
    <p class="sidebar-footer">Belajar dari request pertama sampai state
di React.<br>Data latihan: pengguna fiktif.</p>
  </aside>

  <header class="topbar">
    <span>Modul 07 <span class="context">/ Soal 02: Registrasi Peserta
React</span></span>
    <a href="../../Modul_07_AJAX_Sederhana.pdf">Baca modul PDF</a>
```

```
  </header>

  <div class="mobile-nav">
    <label class="skip" for="pindah-materi">Pilih materi</label>
    <select id="pindah-materi" aria-label="Pindah materi">
      <option value="../../index.html">Peta materi</option>
      <option value="../../01-apa-itu-ajax/index.html">01 Apa itu
AJAX?</option>
      <option value="../../02-cara-kerja-ajax/index.html">02 Cara kerja
AJAX</option>
      <option value="../../03-event-xhr/index.html">03 Event dan
XMLHttpRequest</option>
      <option value="../../04-fetch-get/index.html">04 GET dengan
Fetch</option>
      <option value="../../05-axios-get/index.html">05 GET dengan
Axios</option>
      <option value="../../06-axios-post/index.html">06 POST dengan
Axios</option>
      <option value="../../07-loading-error/index.html">07 Loading dan
error handling</option>
      <option value="../../08-react-state/index.html">08 Loading dan
error di React</option>
      <option value="../../09-api-eksternal/index.html">09 API lokal dan
API eksternal</option>
      <option value="../soal-01/index.html">Soal 01 - Portal
Praktikum</option>
      <option value="index.html" selected>Soal 02 - Registrasi
React</option>
    </select>
  </div>

  <main id="utama" class="sheet">
    <div class="file-warning" data-file-warning hidden>
      <strong>Jalankan server terlebih dahulu.</strong><br>
      Ekstrak seluruh ZIP, lalu klik dua kali <code>JALANKAN.bat</code>.
Buka <a href="http://127.0.0.1:3007">http://127.0.0.1:3007</a>.
      Contoh AJAX perlu alamat HTTP, bukan file://. Node.js 18+
diperlukan; Node.js LTS disarankan.
    </div>

    <div class="page-meta">Tugas Praktikum Modul 07 / Soal 02 (50
Poin)</div>
    <h1>Aplikasi Registrasi Peserta React</h1>
    <p class="intro">Menggunakan Axios (GET &amp;amp; POST), useState,
useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect
```

```
(cleanup AbortController), serta validasi formulir di React.</p>

    <div class="workspace">
      <div id="root" style="min-width:0;"></div>

      <section class="explanation">
        <h2>Inti Materi dan Petunjuk Pengujian</h2>
        <p>Aplikasi React ini menggabungkan manajemen state terpusat
dengan konsumsi API melalui library Axios:</p>
        <ol>
          <li><strong>useEffect &amp;amp; AbortController:</strong> GET
daftar peserta dipicu otomatis saat mount dan saat pilihan skenario
berubah. Cleanup effect membatalkan request yang sedang berjalan dan
mencegah response kadaluwarsa menimpa state terbaru.</li>
          <li><strong>Axios GET &amp;amp; 4 Skenario:</strong>
            <ul>
              <li><em>Berhasil:</em> delay 1000 ms, timeout 5000 ms.</li>
              <li><em>Data kosong:</em> mode=empty, delay 1000 ms,
timeout 5000 ms.</li>
              <li><em>Error HTTP:</em> mode=error (HTTP 503), delay 1000
ms, timeout 5000 ms.</li>
              <li><em>Timeout:</em> delay 2500 ms dengan timeout Axios
700 ms, memicu ECONNABORTED sebelum server selesai merespons.</li>
            </ul>
          </li>
          <li><strong>Axios POST &amp;amp; Validasi:</strong> Formulir
pendaftaran memvalidasi input nama (tidak boleh kosong/spasi) dan format
email dengan regex sebelum mengirim JSON. Respons 201 dengan atribut
<code>simulated: true</code> ditampilkan terpisah tanpa mengubah daftar
GET.</li>
        </ol>

        <h3>Skenario Pengujian Soal 2</h3>
        <ul>
          <li><strong>Uji Muat Ulang:</strong> Buka halaman, lalu klik
tombol <em>Muat ulang</em>. Request baru dipicu tanpa reload
halaman.</li>
          <li><strong>Uji Data Kosong &amp;amp; Error HTTP:</strong> Ganti
skenario ke <em>Data kosong</em> lalu <em>Error HTTP</em>. Daftar lama
segera dibersihkan dan status menampilkan pesan sesuai respons.</li>
          <li><strong>Uji Timeout:</strong> Pilih skenario
<em>Timeout</em>. Request dibatalkan setelah 700 ms (sebelum jeda 2500
ms). Setelah itu, pilih kembali <em>Berhasil</em> untuk memastikan
aplikasi pulih.</li>
          <li><strong>Uji Pergantian Cepat:</strong> Ganti pilihan secara
```

```
cepat saat status masih loading; request sebelumnya dibatalkan oleh
cleanup effect dan hanya response terbaru yang ditampilkan.</li>
          <li><strong>Uji Kirim Valid:</strong> Masukkan nama <code>Rina
Praktikan</code> dan email <code>rina@example.com</code> lalu kirim.
Status HTTP 201, ID 101, data sesuai input, dan <code>simulated:
true</code> muncul, sementara daftar GET tetap utuh.</li>
          <li><strong>Uji Validasi:</strong> Coba kirim nama yang hanya
berisi spasi atau email tanpa format valid; formulir menolak pengiriman
dan menampilkan petunjuk perbaikan.</li>
        </ul>
      </section>
    </div>

    <details class="source" data-source="app.jsx">
      <summary>Lihat kode React JSX: app.jsx</summary>
      <p class="meta">File: latihan-praktikan/soal-02/app.jsx. Komponen
React dengan Axios GET/POST, state, dan effect.</p>
      <pre class="source-code"><code>Memuat kode...</code></pre>
    </details>

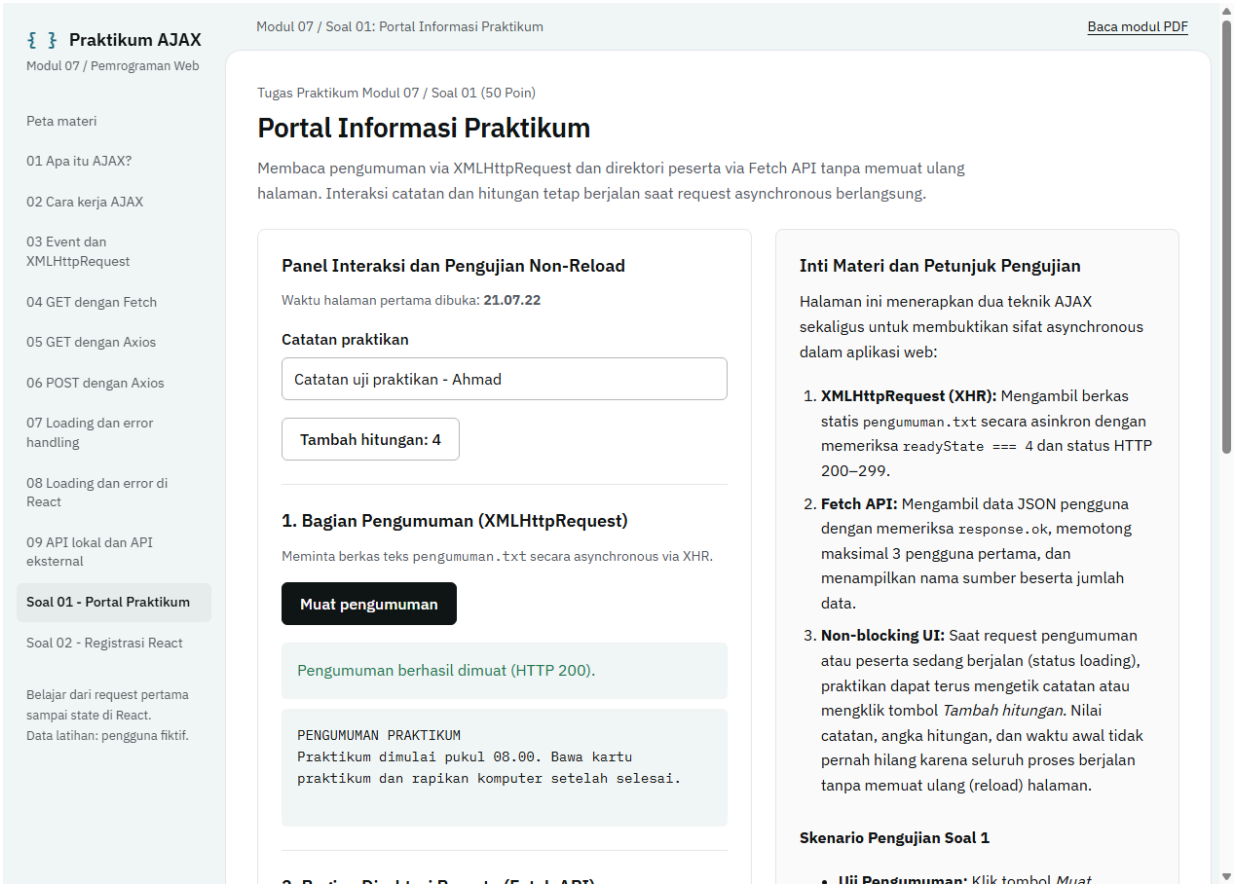
    <details class="source" data-source="index.html">
      <summary>Lihat kode HTML: index.html</summary>
      <pre class="source-code"><code>Memuat kode...</code></pre>
    </details>

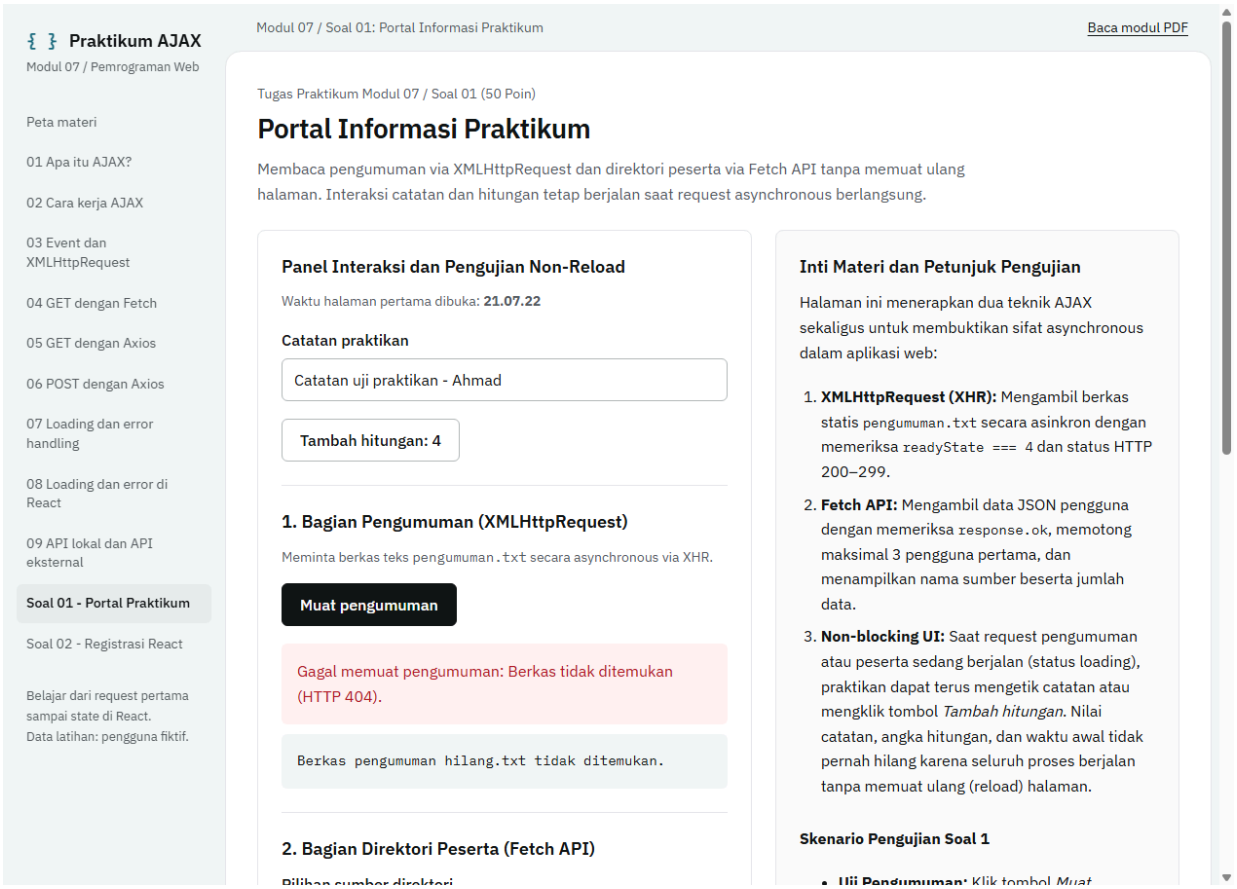
    <div class="pager">
      <a href="../soal-01/">&larr; Kembali ke Soal 01</a>
      <a href="../../index.html">Kembali ke Peta Materi &rarr;</a>
    </div>
  </main>

  <script src="../../assets/ui.js"></script>
  <script src="../../vendor/axios.min.js"></script>
  <script src="../../vendor/react.development.js"></script>
  <script src="../../vendor/react-dom.development.js"></script>
  <script src="../../vendor/babel.min.js"></script>
  <script type="text/babel" data-presets="react" src="app.jsx"></script>
</body>
</html>
```

Hasil Output/SS

Soal 1


{ } Praktikum AJAX
Modul 07 / Pemrograman Web
Peta materi
01 Apa itu AJAX?
02 Cara kerja AJAX
03 Event dan XMLHttpRequest
04 GET dengan Fetch
05 GET dengan Axios
06 POST dengan Axios
07 Loading dan error handling
08 Loading dan error di React
09 API lokal dan API eksternal
Soal 01 - Portal Praktikum
Soal 02 - Registrasi React
Belajar dari request pertama sampai state di React.
Data latihan: pengguna fiktif.
Modul 07 / Soal 01: Portal Informasi Praktikum
Baca modul PDF
Tugas Praktikum Modul 07 / Soal 01 (50 Poin)
Portal Informasi Praktikum
Membaca pengumuman via XMLHttpRequest dan direktori peserta via Fetch API tanpa memuat ulang halaman. Interaksi catatan dan hitungan tetap berjalan saat request asynchronous berlangsung.
Panel Interaksi dan Pengujian Non-Reload
Waktu halaman pertama dibuka: 21.07.22
Catatan praktikan
Catatan uji praktikan - Ahmad
Tambah hitungan: 4
1. Bagian Pengumuman (XMLHttpRequest)
Meminta berkas teks pengumuman.txt secara asynchronous via XHR.
Muat pengumuman
Pengumuman berhasil dimuat (HTTP 200).
PENGUMUMAN PRAKTIKUM
Praktikum dimulai pukul 08.00. Bawa kartu praktikum dan rapikan komputer setelah selesai.
Inti Materi dan Petunjuk Pengujian
Halaman ini menerapkan dua teknik AJAX sekaligus untuk membuktikan sifat asynchronous dalam aplikasi web:
1. XMLHttpRequest (XHR): Mengambil berkas statis pengumuman.txt secara asinkron dengan memeriksa readyState === 4 dan status HTTP 200–299.
2. Fetch API: Mengambil data JSON pengguna dengan memeriksa response.ok, memotong maksimal 3 pengguna pertama, dan menampilkan nama sumber beserta jumlah data.
3. Non-blocking UI: Saat request pengumuman atau peserta sedang berjalan (status loading), praktikan dapat terus mengetik catatan atau mengklik tombol Tambah hitungan. Nilai catatan, angka hitungan, dan waktu awal tidak pernah hilang karena seluruh proses berjalan tanpa memuat ulang (reload) halaman.
Skenario Pengujian Soal 1


{ } Praktikum AJAX
Modul 07 / Pemrograman Web
Peta materi
01 Apa itu AJAX?
02 Cara kerja AJAX
03 Event dan XMLHttpRequest
04 GET dengan Fetch
05 GET dengan Axios
06 POST dengan Axios
07 Loading dan error handling
08 Loading dan error di React
09 API lokal dan API eksternal
Soal 01 - Portal Praktikum
Soal 02 - Registrasi React
Belajar dari request pertama sampai state di React.
Data latihan: pengguna fiktif.
Modul 07 / Soal 01: Portal Informasi Praktikum
Baca modul PDF
Tugas Praktikum Modul 07 / Soal 01 (50 Poin)
Portal Informasi Praktikum
Membaca pengumuman via XMLHttpRequest dan direktori peserta via Fetch API tanpa memuat ulang halaman. Interaksi catatan dan hitungan tetap berjalan saat request asynchronous berlangsung.
Panel Interaksi dan Pengujian Non-Reload
Waktu halaman pertama dibuka: 21.07.22
Catatan praktikan
Catatan uji praktikan - Ahmad
Tambah hitungan: 4
1. Bagian Pengumuman (XMLHttpRequest)
Meminta berkas teks pengumuman.txt secara asynchronous via XHR.
Muat pengumuman
Gagal memuat pengumuman: Berkas tidak ditemukan (HTTP 404).
Berkas pengumuman hilang.txt tidak ditemukan.
Inti Materi dan Petunjuk Pengujian
Halaman ini menerapkan dua teknik AJAX sekaligus untuk membuktikan sifat asynchronous dalam aplikasi web:
1. XMLHttpRequest (XHR): Mengambil berkas statis pengumuman.txt secara asinkron dengan memeriksa readyState === 4 dan status HTTP 200–299.
2. Fetch API: Mengambil data JSON pengguna dengan memeriksa response.ok, memotong maksimal 3 pengguna pertama, dan menampilkan nama sumber beserta jumlah data.
3. Non-blocking UI: Saat request pengumuman atau peserta sedang berjalan (status loading), praktikan dapat terus mengetik catatan atau mengklik tombol Tambah hitungan. Nilai catatan, angka hitungan, dan waktu awal tidak pernah hilang karena seluruh proses berjalan tanpa memuat ulang (reload) halaman.
Skenario Pengujian Soal 1

Soal 2

Tugas Praktikum Modul 07 / Soal 02 (50 Poin)

Aplikasi Registrasi Peserta React

Menggunakan Axios (GET & POST), useState, useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect (cleanup AbortController), serta validasi formulir di React.

1. Daftar Peserta Praktikum (GET Axios)

Pilih skenario response

Timeout (/api/users?delay=2500 - Timeout 700ms)

Muat ulang

Timeout: Batas waktu tunggu (700 ms) terlampaui sebelum server merespons.

2. Formulir Registrasi Peserta (POST Axios)

Nama Peserta

Rina Praktikan

Email Peserta

rina@example.com

Kirim pendaftaran

Hasil Response POST:

Inti Materi dan Petunjuk Pengujian

Aplikasi React ini menggabungkan manajemen state terpusat dengan konsumsi API melalui library Axios:

1. **useEffect & AbortController:** GET daftar peserta dipicu otomatis saat mount dan saat pilihan skenario berubah. Cleanup effect membatalkan request yang sedang berjalan dan mencegah response kadaluwarsa menimpa state terbaru.
2. **Axios GET & 4 Skenario:**
   - *Berhasil:* delay 1000 ms, timeout 5000 ms.
   - *Data kosong:* mode=empty, delay 1000 ms, timeout 5000 ms.
   - *Error HTTP:* mode=error (HTTP 503), delay 1000 ms, timeout 5000 ms.
   - *Timeout:* delay 2500 ms dengan timeout Axios 700 ms, memicu ECONNABORTED sebelum server selesai merespons.
3. **Axios POST & Validasi:** Formulir pendaftaran memvalidasi input nama (tidak boleh

{ } Praktikum AJAX
Modul 07 / Pemrograman Web

Peta materi
01 Apa itu AJAX?
02 Cara kerja AJAX
03 Event dan XMLHttpRequest
04 GET dengan Fetch
05 GET dengan Axios
06 POST dengan Axios
07 Loading dan error handling
08 Loading dan error di React
09 API lokal dan API eksternal
Soal 01 - Portal Praktikum
Soal 02 - Registrasi React

Belajar dari request pertama sampai state di React.
Data latihan: pengguna fiktif.

Modul 07 / Soal 02: Registrasi Peserta React
Baca modul PDF

Tugas Praktikum Modul 07 / Soal 02 (50 Poin)

Aplikasi Registrasi Peserta React

Menggunakan Axios (GET & POST), useState, useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect (cleanup AbortController), serta validasi formulir di React.

1. Daftar Peserta Praktikum (GET Axios)

Pilih skenario response

Error HTTP (/api/users?mode=error&delay=1000 - Timeo

Muat ulang

Gagal: Server merespons HTTP 503 (Simulasi server bermasalah.).

2. Formulir Registrasi Peserta (POST Axios)

Nama Peserta

Rina Praktikan

Email Peserta

rina@example.com

Kirim pendaftaran

Hasil Response POST:

Inti Materi dan Petunjuk Pengujian

Aplikasi React ini menggabungkan manajemen state terpusat dengan konsumsi API melalui library Axios:

1. useEffect & AbortController: GET daftar peserta dipicu otomatis saat mount dan saat pilihan skenario berubah. Cleanup effect membatalkan request yang sedang berjalan dan mencegah response kadaluwarsa menimpa state terbaru.
2. Axios GET & 4 Skenario:
   - Berhasil: delay 1000 ms, timeout 5000 ms.
   - Data kosong: mode=empty, delay 1000 ms, timeout 5000 ms.
   - Error HTTP: mode=error (HTTP 503), delay 1000 ms, timeout 5000 ms.
   - Timeout: delay 2500 ms dengan timeout Axios 700 ms, memicu ECONNABORTED sebelum server selesai merespons.
3. Axios POST & Validasi: Formulir pendaftaran memvalidasi input nama (tidak boleh

Tugas Praktikum Modul 07 / Soal 02 (50 Poin)

Aplikasi Registrasi Peserta React

Menggunakan Axios (GET & POST), useState, useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect (cleanup AbortController), serta validasi formulir di React.

1. Daftar Peserta Praktikum (GET Axios)

Pilih skenario response

Data kosong (/api/users?mode=empty&delay=1000 - Tim

Muat ulang

Data kosong (tidak ada peserta).

2. Formulir Registrasi Peserta (POST Axios)

Nama Peserta

Rina Praktikan

Email Peserta

rina@example.com

Kirim pendaftaran

Hasil Response POST:

Status HTTP: 201 Created

Inti Materi dan Petunjuk Pengujian

Aplikasi React ini menggabungkan manajemen state terpusat dengan konsumsi API melalui library Axios:

1. **useEffect & AbortController:** GET daftar peserta dipicu otomatis saat mount dan saat pilihan skenario berubah. Cleanup effect membatalkan request yang sedang berjalan dan mencegah response kadaluwarsa menimpa state terbaru.
2. **Axios GET & 4 Skenario:**
   - *Berhasil:* delay 1000 ms, timeout 5000 ms.
   - *Data kosong:* mode=empty, delay 1000 ms, timeout 5000 ms.
   - *Error HTTP:* mode=error (HTTP 503), delay 1000 ms, timeout 5000 ms.
   - *Timeout:* delay 2500 ms dengan timeout Axios 700 ms, memicu ECONNABORTED sebelum server selesai merespons.
3. **Axios POST & Validasi:** Formulir pendaftaran memvalidasi input nama (tidak boleh

{ } Praktikum AJAX
Modul 07 / Pemrograman Web

Peta materi
01 Apa itu AJAX?
02 Cara kerja AJAX
03 Event dan XMLHttpRequest
04 GET dengan Fetch
05 GET dengan Axios
06 POST dengan Axios
07 Loading dan error handling
08 Loading dan error di React
09 API lokal dan API eksternal
Soal 01 - Portal Praktikum
Soal 02 - Registrasi React

Belajar dari request pertama sampai state di React.
Data latihan: pengguna fiktif.

Modul 07 / Soal 02: Registrasi Peserta React
Baca modul PDF

Tugas Praktikum Modul 07 / Soal 02 (50 Poin)

Aplikasi Registrasi Peserta React

Menggunakan Axios (GET & POST), useState, useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect (cleanup AbortController), serta validasi formulir di React.

1. Daftar Peserta Praktikum (GET Axios)

Pilih skenario response

Berhasil (/api/users?delay=1000 - Timeout 5000ms)

Muat ulang

Berhasil memuat 3 pengguna.

Budi Santoso — budi@example.com (ID: 1)

Siti Rahma — siti@example.com (ID: 2)

Andi Pratama — andi@example.com (ID: 3)

2. Formulir Registrasi Peserta (POST Axios)

Nama Peserta

Rina Praktikan

Email Peserta

Inti Materi dan Petunjuk Pengujian

Aplikasi React ini menggabungkan manajemen state terpusat dengan konsumsi API melalui library Axios:

1. useEffect & AbortController: GET daftar peserta dipicu otomatis saat mount dan saat pilihan skenario berubah. Cleanup effect membatalkan request yang sedang berjalan dan mencegah response kadaluwarsa menimpa state terbaru.
2. Axios GET & 4 Skenario:
   - Berhasil: delay 1000 ms, timeout 5000 ms.
   - Data kosong: mode=empty, delay 1000 ms, timeout 5000 ms.
   - Error HTTP: mode=error (HTTP 503), delay 1000 ms, timeout 5000 ms.
   - Timeout: delay 2500 ms dengan timeout Axios 700 ms, memicu ECONNABORTED sebelum server selesai merespons.
3. Axios POST & Validasi: Formulir pendaftaran memvalidasi input nama (tidak boleh

Tugas Praktikum Modul 07 / Soal 02 (50 Poin)

Aplikasi Registrasi Peserta React

Menggunakan Axios (GET & POST), useState, useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect (cleanup AbortController), serta validasi formulir di React.

1. Daftar Peserta Praktikum (GET Axios)

Pilih skenario response

Berhasil (/api/users?delay=1000 - Timeout 5000ms)

Muat ulang

Berhasil memuat 3 pengguna.

Budi Santoso — budi@example.com (ID: 1)

Siti Rahma — siti@example.com (ID: 2)

Andi Pratama — andi@example.com (ID: 3)

2. Formulir Registrasi Peserta (POST Axios)

Nama Peserta

Email Peserta

Inti Materi dan Petunjuk Pengujian

Aplikasi React ini menggabungkan manajemen state terpusat dengan konsumsi API melalui library Axios:

1. **useEffect & AbortController:** GET daftar peserta dipicu otomatis saat mount dan saat pilihan skenario berubah. Cleanup effect membatalkan request yang sedang berjalan dan mencegah response kadaluwarsa menimpa state terbaru.
2. **Axios GET & 4 Skenario:**
   - *Berhasil:* delay 1000 ms, timeout 5000 ms.
   - *Data kosong:* mode=empty, delay 1000 ms, timeout 5000 ms.
   - *Error HTTP:* mode=error (HTTP 503), delay 1000 ms, timeout 5000 ms.
   - *Timeout:* delay 2500 ms dengan timeout Axios 700 ms, memicu ECONNABORTED sebelum server selesai merespons.
3. **Axios POST & Validasi:** Formulir pendaftaran memvalidasi input nama (tidak boleh

{ } Praktikum AJAX
Modul 07 / Pemrograman Web

Peta materi
01 Apa itu AJAX?
02 Cara kerja AJAX
03 Event dan XMLHttpRequest
04 GET dengan Fetch
05 GET dengan Axios
06 POST dengan Axios
07 Loading dan error handling
08 Loading dan error di React
09 API lokal dan API eksternal
Soal 01 - Portal Praktikum
Soal 02 - Registrasi React

Belajar dari request pertama sampai state di React.
Data latihan: pengguna fiktif.

Modul 07 / Soal 02: Registrasi Peserta React
Baca modul PDF

Tugas Praktikum Modul 07 / Soal 02 (50 Poin)

Aplikasi Registrasi Peserta React

Menggunakan Axios (GET & POST), useState, useEffect, pengelolaan kondisi loading/error/timeout, pembersihan effect (cleanup AbortController), serta validasi formulir di React.

1. Daftar Peserta Praktikum (GET Axios)

Pilih skenario response

Berhasil (/api/users?delay=1000 - Timeout 5000ms)

Muat ulang

Berhasil memuat 3 pengguna.

Budi Santoso — budi@example.com (ID: 1)

Siti Rahma — siti@example.com (ID: 2)

Andi Pratama — andi@example.com (ID: 3)

2. Formulir Registrasi Peserta (POST Axios)

Nama Peserta

Email Peserta

Inti Materi dan Petunjuk Pengujian

Aplikasi React ini menggabungkan manajemen state terpusat dengan konsumsi API melalui library Axios:

1. useEffect & AbortController: GET daftar peserta dipicu otomatis saat mount dan saat pilihan skenario berubah. Cleanup effect membatalkan request yang sedang berjalan dan mencegah response kadaluwarsa menimpa state terbaru.
2. Axios GET & 4 Skenario:
   - Berhasil: delay 1000 ms, timeout 5000 ms.
   - Data kosong: mode=empty, delay 1000 ms, timeout 5000 ms.
   - Error HTTP: mode=error (HTTP 503), delay 1000 ms, timeout 5000 ms.
   - Timeout: delay 2500 ms dengan timeout Axios 700 ms, memicu ECONNABORTED sebelum server selesai merespons.
3. Axios POST & Validasi: Formulir pendaftaran memvalidasi input nama (tidak boleh